

MARST: Code Explanation

Companion to the Thesis Article

This document walks through the MARST implementation that produced the results reported in the thesis article. It is organised to match the structure of Section 3 (*The MARST Model*) so that each architectural component in the paper has a direct code counterpart here. All code excerpts are taken verbatim from the reference implementation in `marst-multigpu.ipynb`. Section numbers in parentheses refer back to the paper.

1. Implementation Overview

The full MARST pipeline is implemented in a single Jupyter notebook. The notebook is structured into 48 cells, but only four cells contain the architecturally meaningful code:

Cell	Contents	Approx. lines
3	Imports, configuration constants, dataset registry, data-loading helpers	200
4	<code>STBlock</code> and <code>MaskedSTTransformer</code> MARST model classes	165
13	The <code>run_dataset(name)</code> per-dataset pipeline (data preparation, baselines, MARST training, evaluation, leak audit, sparsity sweep, ablations, JSON save)	1700
15	The driver loop that calls <code>run_dataset</code> for each of the four traffic datasets	15

The remaining cells contain markdown and downstream analysis (figures, statistical tests, aggregation across datasets). They do not affect the model.

This document focuses on cells 3, 4, and the model-relevant parts of cell 13.

2. Configuration Constants (Cell 3)

The first executable block of the notebook fixes the experimental protocol. Each constant has a one-to-one correspondence with a paper claim, so we reproduce them verbatim:

```

GLOBAL_SEED      = 42
SPARSITY         = 0.80                                # fraction of sensors blind at evaluation
BATCH_TIME       = 48                                  # 4-hour window at 5-minute sampling
HIDDEN_DIM_MARST = 128                                 # transformer hidden width
N_LAYERS_MARST   = 6                                   # stacked spatiotemporal blocks
N_HEADS          = 4                                   # attention heads per layer
DROPOUT          = 0.1
TRAIN_EPOCHS     = 800                                 # main training duration
TRAIN_SEEDS      = [0, 1, 2]                           # three independent training runs
EVAL_SEEDS       = [42, 43, 44, 45, 46]                # five independent eval-mask seeds
STEPS_PER_DAY    = 288                                 # 24 h / 5 min for traffic; 96 for electricity
WINDOW          = 5000                                 # timesteps per dataset
TRAIN_END        = 4000                                 # train window ends here
EVAL_START       = 4500                                 # 500-step anti-leak gap
EVAL_LEN         = 450
JAM_WEIGHT       = 1.0                                 # 2x loss weight on congested positions
HUBER_BETA       = 1.0                                 # Huber transition point

```

The 500-timestep gap between `TRAIN_END` and `EVAL_START` is the **temporal anti-leak invariant** discussed in Section 3.4 of the paper. Without this gap, the LOCF anchor at the start of the evaluation window would carry forward the last training-window value, contaminating the comparison.

The `DATASETS` dictionary that follows registers per-dataset metadata (data files, adjacency files, source URLs, regime thresholds). For traffic datasets, the entry follows the pattern:

```

'PEMS-BAY': dict(kind='speed', steps_per_day=288, unit='mph',
                  data_files=['pems-bay.h5', 'PEMS-BAY.h5', ...],
                  adj_files=['adj_mx_bay.pkl', ...],
                  data_url='https://zenodo.org/.../PEMS-BAY.csv?download=1',
                  adj_url='https://zenodo.org/.../adj_mx_bay.pkl?download=1'),

```

The `kind` field switches the data-loading branch in `load_raw_array` (different file formats per dataset family) and the regime-edge convention in `extended_metrics` (slowest 20 % for speed datasets, busiest 33 % for flow datasets).

3. The Spatiotemporal Transformer Block (Cell 4)

The `STBlock` class implements a single layer of the spatiotemporal transformer trunk (Section 3.3 of the paper). It alternates a causal-temporal attention sub-layer with a mask-aware spatial attention sub-layer, followed by a position-wise feed-forward network.

```

class STBlock(nn.Module):
    def __init__(self, hidden, n_heads, ff_mult=2, dropout=0.1):
        super().__init__()
        self.t_attn = nn.MultiheadAttention(hidden, n_heads,
                                             dropout=dropout, batch_first=True)
        self.s_attn = nn.MultiheadAttention(hidden, n_heads,
                                             dropout=dropout, batch_first=True)
        self.ff      = nn.Sequential(
            nn.Linear(hidden, ff_mult * hidden), nn.GELU(),
            nn.Linear(ff_mult * hidden, hidden))
        self.ln1, self.ln2, self.ln3 = (nn.LayerNorm(hidden) for _ in range(3))
        self.drop  = nn.Dropout(dropout)

```

The forward pass treats the input tensor of shape `[B, N, T, H]` (batch, nodes, time, hidden) as a stack of temporal sequences for the time-attention sub-layer and as a stack of spatial sequences for the spatial sub-layer. **Causality is enforced** in the temporal sub-layer by a triangular mask:

```
def forward(self, x, spatial_pad_mask=None):
    B, N, T, H = x.shape

    # Temporal sub-layer: causal self-attention along T
    h = x.reshape(B * N, T, H)
    mask = torch.triu(torch.full((T, T), float('-inf'), device=x.device),
                      diagonal=1)
    t_out, _ = self.t_attn(h, h, h, attn_mask=mask, need_weights=False)
    x = self.ln1(x + self.drop(t_out).reshape(B, N, T, H))

    # Spatial sub-layer: self-attention across N, with mask-aware padding
    h = x.permute(0, 2, 1, 3).reshape(B * T, N, H)
    s_out, _ = self.s_attn(h, h, h, key_padding_mask=spatial_pad_mask,
                          need_weights=False)
    x = self.ln2(x + self.drop(s_out).reshape(B, T, N, H).permute(0, 2, 1, 3))

    # Feed-forward
    x = self.ln3(x + self.drop(self.ff(x)))
    return x
```

Two technical notes:

1. **Causal mask.** `torch.triu(..., diagonal=1)` produces an upper-triangular mask with `-inf` above the diagonal. When added to attention logits this prevents each position t from attending to positions $s > t$. This is the architectural realisation of the streaming constraint in Section 3.1.
2. **Spatial padding mask.** `spatial_pad_mask` has shape `[B*T, N]` and marks blind sensors at the current timestep. The spatial attention is allowed to attend to currently-observed sensors only. This is what makes the spatial sub-layer *mask-aware* in the sense used throughout the paper.

4. Causal Anchor Computation (Cell 4)

The `_compute_causal_signals` method of `MaskedSTTransformerMARST` constructs the LOCF anchor and the staleness feature in a single pass through time:

```

def _compute_causal_signals(self, x, m, ha_prior):
    B, N, T = x.shape
    locf = torch.zeros_like(x)
    staleness = torch.zeros_like(x)
    soft_locf = torch.zeros_like(x)
    cur_locf = ha_prior[:, :, 0].clone() # HA-initialisation, not zero
    cur_stale = torch.zeros(B, N, device=x.device)
    cur_soft = torch.zeros(B, N, device=x.device)
    decay = self.soft_locf_decay

    for t in range(T):
        obs_t = x[:, :, t]
        mask_t = m[:, :, t]
        ha_t = ha_prior[:, :, t]
        obs = mask_t > 0.5

        # Hard LOCF: carry forward last observed value
        cur_locf = torch.where(obs, obs_t, cur_locf)
        # Staleness: steps since last observation, normalised by BATCH_TIME
        cur_stale = torch.where(obs, torch.zeros_like(cur_stale),
                                cur_stale + 1.0)
        # Soft LOCF: EMA toward HA prior (decay-controlled, see paper §5.4)
        cur_soft = torch.where(obs, obs_t,
                                decay * cur_soft + (1.0 - decay) * ha_t)

        locf[:, :, t] = cur_locf
        staleness[:, :, t] = cur_stale / 48.0
        soft_locf[:, :, t] = cur_soft

    return locf, staleness, soft_locf

```

Two design choices in this loop are load-bearing for the headline numbers:

- **HA-initialisation of `cur_locf`** (line `cur_locf = ha_prior[:, :, 0].clone()`). At the start of the eval window, no sensor has been "previously observed" inside the window. Initialising LOCF to the HA prior produces a sensible value for sensors that have been dark since the start of the window. Initialising to zero (the original default in some baselines) makes the first-window error dramatically worse.
- **The 500-step `TRAIN_END` → `EVAL_START` gap** is what gives the HA initialisation its anti-leak meaning. With the gap, `ha_prior[t, n]` is computed from training-window-only data and cannot contain held-out truth.

The KNN anchor is computed by a separate method:

```

def _knn_anchor(self, x, m, ha_prior):
    mean_v = self.node_means.view(1, -1, 1)
    std_v = self.node_stds.view(1, -1, 1)
    x_kmh = x * std_v + mean_v # de-z-score per node
    num = torch.matmul(self.adj_static, x_kmh * m) # observed neighbour sum
    den = torch.matmul(self.adj_static, m) # observed neighbour count
    knn_z = (num / (den + 1e-6) - mean_v) / std_v # z-score by target node
    return torch.where(den > 0, knn_z, ha_prior) # HA fall-back

```

The `self.adj_static` buffer is the binary road adjacency with the diagonal explicitly zeroed (the **spatial anti-leak invariant**). Without the zero diagonal, sensor n 's own value would appear in its own neighbour mean, which would directly leak the held-out truth into the KNN anchor at observed timesteps.

5. The Forward Pass (Cell 4)

The `forward` method of `MaskedSTTransformerMARST` produces the final imputed value by composing the three anchors through the learned softmax mixture and the residual head:

```
def forward(self, x, m, t_sin, t_cos, ha_prior):
    B, N, T = x.shape
    mean_v = self.node_means.view(1, -1, 1)
    std_v = self.node_stds.view(1, -1, 1)

    # (a) Compute the three causal anchors
    locf, staleness, soft_locf = self._compute_causal_signals(x, m, ha_prior)
    locf_kmh = locf * std_v + mean_v
    a_locf = locf
    a_ha = ha_prior
    a_knn = self._knn_anchor(x, m, ha_prior)

    # (b) Extra spatial-context feature: 2-hop neighbour mean of LOCF
    n_mean_2hop = (torch.matmul(self.adj_2hop, locf_kmh) - mean_v) / std_v

    # (c) Build the 9-channel feature stack
    feat = torch.stack([x, m, t_sin, t_cos, a_locf, a_ha, a_knn,
                       staleness, n_mean_2hop], dim=-1)
    h = self.in_proj(feat)

    # (d) Add per-sensor identity embedding and time-of-day position embedding
    h = h + self.node_emb.view(1, N, 1, self.hidden)
    h = h + self.pos_emb[:T].view(1, 1, T, self.hidden)

    # (e) Six STBlock layers with causal-temporal + mask-aware-spatial attention
    spatial_pad_mask = (m == 0).permute(0, 2, 1).contiguous().view(B * T, N)
    for blk in self.blocks:
        h = blk(h, spatial_pad_mask=spatial_pad_mask)
    h = self.final_norm(h)

    # (f) Learned anchor mixture  $\pi$  (3-simplex per position)
    pi = torch.softmax(self.anchor_gate(h), dim=-1) # [B, N, T, 3]
    anchors = torch.stack([a_locf, a_ha, a_knn], dim=-1) # [B, N, T, 3]
    blended = (pi * anchors).sum(dim=-1) # [B, N, T]
    self.last_pi = pi.detach().reshape(-1, 3) # diagnostic

    # (g) Residual correction gated by alpha (meta-gate)
    residual = self.residual_head(h).squeeze(-1)
    adj_m_obs = torch.matmul(self.adj_static, m)
    n_obs = adj_m_obs / (self.adj_static.sum(dim=-1, keepdim=True) + 1e-8)
    alpha = self.meta_gate(torch.cat([h, n_obs.unsqueeze(-1)], dim=-1)).squeeze(-1)
    self.last_alpha = alpha.detach()

    return blended + alpha * residual
```

The seven labelled blocks in the code correspond directly to the equations in Section 3.3 of the paper:

- (a) computes the three causal anchors a_{LOCF} , a_{HA} , a_{KNN} .
- (b) computes an additional 2-hop neighbour-mean feature used as input context but *not* as an anchor.
- (c)–(d) assemble the 9-channel feature stack and project it to the 128-dimensional hidden representation, augmented with the learnable per-sensor and time-of-day embeddings.
- (e) runs the stack through six STBlock layers.
- (f) emits the softmax mixture π and the blended anchor.
- (g) computes the residual r and the meta-gate α , returning the final imputation.

The detached `self.last_pi` and `self.last_alpha` attributes are recorded purely for downstream diagnostic inspection (the interpretability tables in Section 5.5 of the paper). They are not used by the loss.

6. The Training Loop (Cell 13)

The `_train_st` function trains one MARST instance for a given training seed. It encapsulates four design choices reported in the paper:

1. The **sparsity curriculum** (Section 3.4): the training mask sparsity ramps from 60 % to the target 80 % over the first 75 % of epochs.
2. The **jam-weighted Huber loss** (Section 3.4): congested positions get $2\times$ weight.
3. Per-thread RNG state for **concurrent multi-seed execution** across two GPUs (Section 4, Hardware).
4. A per-epoch fresh **random observation mask** drawn on the device.

The function body is reproduced in condensed form:

```

def _train_st(ctor, epochs, batch_size, train_seed, label,
              use_curriculum=True, gpu_id=0):
    dev = torch.device(f'cuda:{gpu_id}') if torch.cuda.is_available() \
        else torch.device('cpu')

    if torch.cuda.is_available():
        torch.cuda.set_device(dev)

    # Per-GPU data mirrors. .to(dev) is a no-op when already on dev.
    sg, vg, hp = speed_gpu.to(dev), valid_gpu.to(dev), ha_prior.to(dev)
    nm_t, ns_t = node_means_t.to(dev), node_stds_t.to(dev)

    # Per-thread RNGs avoid global numpy / torch RNG races across workers
    rng_np = np.random.default_rng(train_seed)
    g_cuda = torch.Generator(device=dev)
    g_cuda.manual_seed(int(train_seed) * 31 + int(gpu_id))
    torch.manual_seed(train_seed)

    net = ctor().to(dev)
    opt = torch.optim.Adam(net.parameters(), lr=1e-3)
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)

    curr_end = max(1, int(epochs * 0.75)) # curriculum endpoint

    for ep in range(1, epochs + 1):
        net.train()
        if use_curriculum:
            sparsity_ep = min(SPARSITY,
                             0.60 + (SPARSITY - 0.60)
                             * min(1.0, (ep - 1) / curr_end))
        else:
            sparsity_ep = SPARSITY

        # Sample a fresh batch of training windows
        t0_list = rng_np.integers(0, TRAIN_END - BATCH_TIME, batch_size)
        xs, has, ss, cs, ms, vs = [], [], [], [], [], []
        for t0 in t0_list:
            xs.append(sg[t0:t0+BATCH_TIME].T)
            has.append(hp[t0:t0+BATCH_TIME].T)
            vs.append(vg[t0:t0+BATCH_TIME].T)
            ti = torch.arange(int(t0), int(t0) + BATCH_TIME, device=dev)
            ss.append(torch.sin(2*np.pi*(ti % STEPS_PER_DAY)/STEPS_PER_DAY)
                      .view(1, -1).expand(NUM_NODES, -1))
            cs.append(torch.cos(2*np.pi*(ti % STEPS_PER_DAY)/STEPS_PER_DAY)
                      .view(1, -1).expand(NUM_NODES, -1))
            ms.append((torch.rand(NUM_NODES, BATCH_TIME, device=dev,
                                   generator=g_cuda) > sparsity_ep).float())
        xb, hb, sb, cb, mb, vb = map(torch.stack,
                                       (xs, has, ss, cs, ms, vs))

        m_eff = mb * vb

        # Forward + jam-weighted Huber loss
        p = net(xb * m_eff, m_eff, sb, cb, hb)
        lm = (mb == 0) & (vb > 0)
        if not lm.any():
            continue
        _per_elem = F.smooth_l1_loss(p[lm], xb[lm], beta=HUBER_BETA,
                                     reduction='none')
        _xb_raw = xb * ns_t.view(1, -1, 1) + nm_t.view(1, -1, 1)
        if CFG['kind'] == 'speed':
            _jam = _xb_raw < REGIME_EDGES[0]
        else:
            _jam = _xb_raw > REGIME_EDGES[1]
        _w = 1.0 + JAM_WEIGHT * _jam[lm].float()

```

```

    loss = (_per_elem * _w).sum() / _w.sum().clamp(min=1e-6)

    # Backward + clipped gradient step
    opt.zero_grad(); loss.backward()
    torch.nn.utils.clip_grad_norm_(net.parameters(), 0.5)
    opt.step(); sch.step()

    if torch.cuda.is_available():
        net = net.to(torch.device("cuda:0")) # pin returned model to GPU 0
    return net

```

The `_train_st` function is dispatched concurrently across GPUs by `_train_parallel`, which uses `concurrent.futures.ThreadPoolExecutor(max_workers=N_GPUS)` to launch up to two training jobs in parallel. With three training seeds on two GPUs, seeds 0 and 1 train simultaneously on devices `cuda:0` and `cuda:1`; seed 2 picks up the first device to free.

7. Per-Dataset Pipeline (Cell 13)

The `run_dataset(name)` function orchestrates the full per-dataset experiment. Its high-level structure is:


```

def run_dataset(name):
    DATASET = name
    CFG = DATASETS[DATASET]
    STEPS_PER_DAY = int(CFG.get('steps_per_day', 288))

    # 1. Load data and compute the validity mask (V)
    value_raw = load_raw_array()[:WINDOW]
    if CFG['kind'] == 'speed':
        valid_raw = (value_raw > 0).astype(np.float32)
    else:
        valid_raw = np.ones_like(value_raw, dtype=np.float32)

    # 2. Compute regime edges, clamp range, per-sensor mean/std
    REGIME_EDGES = (np.percentile(vt, 20), np.percentile(vt, 40))
    node_means, node_stds = ... # over training window only
    speed_norm = (value_raw - node_means) / node_stds

    # 3. Compute HA prior per (sensor, slot-of-day) on training window
    for s in range(STEPS_PER_DAY):
        sel = slot_idx[:TRAIN_END] == s
        sums = (speed_norm[:TRAIN_END][sel] * valid_raw[:TRAIN_END][sel]).sum(0)
        cnts = valid_raw[:TRAIN_END][sel].sum(0) + 1e-8
        ha_prior[slot_idx == s] = sums / cnts

    # 4. Move all tensors to GPU
    speed_gpu, valid_gpu, ha_prior = (torch.tensor(x).to(device)
                                       for x in (speed_norm, valid_raw, ha_prior))

    # 5. Build the 5 eval masks
    for seed in EVAL_SEEDS:
        m_TN[seed] = make_eval_mask_np(seed, EVAL_LEN, NUM_NODES,
                                       sparsity=SPARSITY)

    # 6. Train all baselines (18 of them, 3 seeds each) and record MAE
    # ...

    # 7. Train MARST (3 seeds, concurrent on 2 GPUs)
    _marst_nets = _train_parallel([
        dict(ctor=ctor_marst, epochs=TRAIN_EPOCHS,
             batch_size=BATCH_SIZE_MARST, seed=ts, label=f"MARST(seed={ts})")
        for ts in TRAIN_SEEDS])

    # 8. Evaluate MARST on the 5 eval masks
    marst_mask_mat = [_eval_st_mae(net) for net in _marst_nets]

    # 9. Extended metrics + leak audit + sparsity sweep
    # 10. Optional ablations (anchor variants, decay sweep, curriculum)
    # 11. Save results_<DATASET>.json

```

The detailed steps 6–11 are elided here for brevity; their structures are straightforward extensions of the same pattern. The save block writes a JSON containing every metric, every seed, every ablation variant, and every leak-audit pass/fail result so that the downstream aggregation cells (17 and onward) can produce the cross-dataset table without re-training anything.

8. The Leak Audit (Cell 13)

The strict-causality claim in Section 5.7 of the paper is verified by a two-part runtime check. The held-out-truth corruption test perturbs the truth at held-out positions and verifies that predictions are unchanged:

```
# Random perturbation of held-out truth
_xc = _x.clone()
_xc[_held] = _xc[_held] + 50.0 * torch.randn_like(_xc[_held])

# Predict on clean and corrupted inputs; difference at held-out positions
# must be exactly zero for a leak-free model
d = (_pred_st_leak(net, _x)[_held] -
     _pred_st_leak(net, _xc)[_held]).abs().max().item()
print(f" {name:<14} held-out-truth leak: max|dpred|={d:.2e} "
      f"-> {'NO LEAK' if d<1e-4 else 'LEAK!!'}")
```

This loop runs over all 14 baselines plus MARST. On every dataset, every model in the comparison reports `max|dpred| = 0.00e+00 → NO LEAK`. The future-perturbation test for MARST itself perturbs the second half of the eval window and verifies the first half's predictions are unchanged:

```
_k = _EL // 2
_xf = _x.clone()
_xf[:, :, _k:] = _xf[:, :, _k:] + 50.0 * torch.randn_like(_xf[:, :, _k:])
_dc = (_pred_st_leak(net_marst, _x)[:, :, :_k] -
      _pred_st_leak(net_marst, _xf[:, :, :_k]).abs().max().item()
print(f" {'MARST (causal)':<14} future-perturb leak: max|dpast|={_dc:.2e} "
      f"-> {'CAUSAL' if _dc<1e-4 else 'LEAK!!'}")
```

For every dataset MARST reports `max|dpast| = 0.00e+00 → CAUSAL`. The audit then prints the aggregate verdict `AUDIT PASSED: no model reads held-out data; ST model is strictly causal`.

These two checks are not slogans — they are run on every dataset, every time, and assert-fail the pipeline if any model registers a non-zero leak. They are the empirical realisation of the architectural invariants discussed in Sections 3.1 and 3.2 of the paper.

9. The Driver Loop (Cell 15)

The driver loop at cell 15 is short:

```
import os, traceback
DATASETS_TO_RUN = ['PEMS-BAY', 'METR-LA', 'PEMS04', 'PEMS08']
FORCE_RERUN = False

for _ds in DATASETS_TO_RUN:
    _out = f'results_{_ds}.json'
    if os.path.exists(_out) and not FORCE_RERUN:
        print(f'skip {_ds}: {_out} already exists')
        continue
    try:
        run_dataset(_ds)
    except Exception as e:
        print(f"!!! FAILED {_ds}: {e}")
        traceback.print_exc()
print("All requested datasets processed.")
```

Three properties of this loop deserve note:

1. **Resume-from-crash semantics.** The existence check on `results_<DATASET>.json` allows a restarted Kaggle session to skip any dataset that has already been completed in a prior run. The full 4-dataset pipeline is therefore safely interruptible: any work done before the interrupt is preserved.
 2. **Per-dataset isolation.** Each dataset call is wrapped in `try/except`. A failure on one dataset does not abort the others; the loop logs the traceback and proceeds.
 3. **Append-only side effects.** Each dataset writes exactly one JSON file. No previously-written JSON is modified. The cross-dataset aggregation cells therefore work whether the four datasets were run together or one at a time.
-

10. Engineering Notes for Reviewers

Three engineering details that are easy to miss when reading the code but matter for reproducibility.

10.1 Cross-device tensor management

When `_train_st` is dispatched concurrently on two GPUs, the trained model is moved back to `cuda:0` before returning. This is essential because downstream evaluation code accesses module-scope tensors (`speed_gpu`, `ha_prior`, etc.) that live on `cuda:0`:

```
if torch.cuda.is_available():
    net = net.to(torch.device("cuda:0"))
return net
```

The explicit `torch.device("cuda:0")` (rather than the module-scope `device = torch.device('cuda')`) is required because in a worker thread that set its current device to `cuda:1`, the bare `'cuda'` resolves to `cuda:1`, not `cuda:0`. An earlier version of the code used the bare `device` and produced a runtime `RuntimeError: Expected all tensors to be on the same device` on the eval call following concurrent training; the explicit `cuda:0` is the fix.

10.2 Per-thread RNG state

The standard `np.random.seed` and `torch.manual_seed` mutate global RNG state and are not thread-safe. With two concurrent training workers, calls from one worker overwrite the RNG state for the other, breaking seed reproducibility. The code uses:

```
rng_np = np.random.default_rng(train_seed)
g_cuda = torch.Generator(device=dev)
g_cuda.manual_seed(int(train_seed) * 31 + int(gpu_id))
```

Each worker has its own NumPy generator (`rng_np`) and its own CUDA generator (`g_cuda`). All random draws inside the worker use these local generators (`rng_np.integers(...)`, `torch.rand(..., generator=g_cuda)`), not the global RNGs. With this discipline, the two workers' results are deterministic conditional on `train_seed` and `gpu_id`.

10.3 Causal-mask boundary

The causal attention mask in `STBlock.forward` uses `diagonal=1`:

```
mask = torch.triu(torch.full((T, T), float('-inf'), device=x.device),
                    diagonal=1)
```

With `diagonal=1`, position t is allowed to attend to positions $\leq t$. With `diagonal=0` (the off-by-one alternative), position t could only attend to positions $< t$ — which would be over-strict and prevent the model from using the current timestep's own (observed) value. The `diagonal=1` choice is the conventional one for causal self-attention but is worth flagging because the alternative is the more commonly-occurring bug.

11. Mapping Code to Paper Sections

Paper Section	Code Location
3.1 Problem setup (notation)	Cell 3 (constants), Cell 13 lines 1–30
3.2 Three causal anchors	Cell 4 <code>_compute_causal_signals</code> , <code>_knn_anchor</code> ; Cell 13 lines 40–110 (HA prior computation)
3.3 Architecture (STBlock + MARST forward)	Cell 4 <code>STBlock</code> , <code>MaskedSTTransformerMARST.forward</code>
3.4 Training: jam-weighted Huber + curriculum	Cell 13 <code>_train_st</code> lines 35–90
4 Experimental setup	Cell 3 (<code>DATASETS</code> , hyperparameters), Cell 13 (preprocessing)
5.1 Main MAE table	Cell 13 baseline + MARST training loops; downstream aggregation cells 17, 35
5.2 JamMAE	Cell 13 <code>extended_metrics</code> function; "MAE leader = JamMAE leader" check at end of extended-metrics block
5.3 Sparsity sweep	Cell 13 <code>_eval_at_sparsity</code> and the <code>SPARSITY_LEVELS</code> loop
5.4 Ablations	Cell 13 <code>MARST_ABLATIONS</code> dict, <code>_train_parallel</code> dispatch, decay sweep, curriculum-vs-fixed
5.5 Anchor mixture interpretability	Cell 4 <code>self.last_pi</code> assignment; Cell 13 anchor-mix print at end of training
5.6 Cross-domain validation	Cell 3 <code>DATASETS['ELECTRICITY']</code> , Cell 13 <code>_load_electricity</code> and <code>_correlation_adjacency</code> helpers
5.7 Leak audit	Cell 13 leak-audit block (after extended metrics)

12. Total Lines of Code by Component

Component	Lines
Configuration and dataset registry	~70
Data loaders (with downloader, file finder, validity mask)	~130
Normalisation and HA prior computation	~50
STBlock	~30
MaskedSTTransformerMARST (model class)	~135
_train_st (training loop)	~75
_train_parallel (multi-GPU dispatcher)	~25
_eval_st_mae (evaluation)	~50
Leak audit	~80
Extended metrics	~70
Sparsity sweep	~60
Anchor ablation block	~50
Save-to-JSON block	~40
18 baseline implementations	~600
Total core code (excluding figures)	~1,500

The 18 baseline implementations make up the largest single chunk because each baseline (LSTM, GRU, TCN, BRITS, DCRNN, GWN, STID, iTransformer, etc.) is implemented from scratch as a small class with its own `__init__` and `forward`. This keeps the comparison apples-to-apples — every model is trained under the same protocol with the same loss, the same optimiser, the same epoch budget, and the same hyperparameter discipline.

The reference implementation is `marst-multigpu.ipynb` on the project GitHub repository.